

GETMATCHREADY × ZITE

Wire training into your **assignment pipeline**

Two calls connect your system to GetMatchReady: send an official in with a signed **launch link**, and receive a signed **completion webhook** when they finish. A third call lets you **pull status** any time. No JavaScript to write, no SDK to install — standard JWT + HMAC your team already has libraries for (.NET included).

THE LOOP

How a training requirement clears, end to end

- 1 An official applies to an event and is accepted in Zite.
- 2 Zite builds a **signed launch link** and sends the official to the required GetMatchReady course.
- 3 They complete it — real watch time, pass/fail, a score — and earn their certificate.
- 4 GetMatchReady fires a **signed completion webhook** back to your endpoint.
- 5 Zite marks the requirement done; assigners and event managers see who's cleared — automatically.

Base URL <https://YOUR-GMR-HOST>

The same contract serves every organization — only the **org** value and credentials change per org. Jeremy provides the base URL and the two credentials (below) for each org you connect (e.g.

STEP 1 • INBOUND

ZITE → GETMATCHREADY

Launch link — drop a specific official into a course

A standard **JWT (HS256)** signed with the shared secret. Put it on the query string of the launch URL. Mint it with any JWT library.

Endpoint	GET {BASE}/launch?token=<JWT>
Algorithm	HS256, signed with the shared secret (INTEGRATION_SECRET)
Claims	refId (their ID in your system) · name · email · moduleId (the course ID) · org · callbackUrl (<i>optional — override where this one completion is posted</i>) · plus standard iat/exp
Expiry	Set exp as you like (minutes for a one-click launch, or days for an emailed link).

MINT A LINK (NODE)

```
// npm i jsonwebtoken
const jwt = require('jsonwebtoken');
const token = jwt.sign({
  refId: 'OFFICIAL-12345',      // your ID for this person
  name: 'Jane Official',
  email: 'jane@example.com',
  moduleId: 'crs_abc123',      // the GetMatchReady course ID
  org: 'omg'
}, SHARED_SECRET, { algorithm: 'HS256', expiresIn: '7d' });

const launchUrl = `${BASE}/launch?token=${token}`; // send the official
here
```

MINT A LINK (.NET / C#)

```
// System.IdentityModel.Tokens.Jwt
var claims = new[] {
  new Claim("refId", "OFFICIAL-12345"),
```

```

new Claim("email", "jane@example.com"),
new Claim("moduleId", "crs_abc123"),
new Claim("org", "omg")
};
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(SHARED_SECRET));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
var jwt = new JwtSecurityToken(claims: claims, expires:
DateTime.UtcNow.AddDays(7), signingCredentials: creds);
var token = new JwtSecurityTokenHandler().WriteToken(jwt);

```

STEP 2 · OUTBOUND

GETMATCHREADY → ZITE

Completion webhook — we tell you when they finish

When a learner reaches a terminal state, we POST JSON to your endpoint, **signed** so you can trust it. We retry on 5xx so a brief outage never loses a completion.

Your endpoint	An https:// URL you give us (global, or per-launch via the callbackUrl claim)
Method	POST · Content-Type: application/json
Signature	X-GetMatchReady-Signature: sha256=<hex> — HMAC-SHA256 of the raw request body , keyed with the shared secret

EXAMPLE PAYLOAD

```

{
  "event":      "module.passed",      // module.<status>
  "refId":     "OFFICIAL-12345",      // echoes your launch claim
  "moduleId":  "crs_abc123",
  "org":       "omg",
  "status":    "passed",              // passed | failed | completed |
incomplete
  "score":     92,
  "startedAt": "2026-10-04T15:12:00Z",
  "completedAt": "2026-10-04T15:47:30Z",
  "certificateId": "cert_7h2k9",

```

```
"durationSeconds": 2130
}
```

VERIFY THE SIGNATURE BEFORE TRUSTING IT (NODE / EXPRESS)

```
// use the RAW body, not the parsed object
const crypto = require('crypto');
function verify(rawBody, header) {
  const expected = 'sha256=' +
    crypto.createHmac('sha256',
      SHARED_SECRET).update(rawBody).digest('hex');
  return crypto.timingSafeEqual(Buffer.from(header),
    Buffer.from(expected));
}

app.post('/gmr/webhook', express.raw({ type: '*' }), (req, res) => {
  if (!verify(req.body, req.get('X-GetMatchReady-Signature'))) return
  res.sendStatus(401);
  const e = JSON.parse(req.body);
  if (e.status === 'passed' || e.status === 'completed')
  markCleared(e.refId, e.moduleId);
  res.sendStatus(200); // 2xx = received; non-2xx 5xx = we retry
});
```

STEP 3 · ON DEMAND

ZITE → GETMATCHREADY

Pull status & self-test

Reconcile any time (nightly job, or to backfill after an outage), and fire a sample to test your receiver before go-live. These use the **API key** (separate from the webhook secret).

RECONCILE — PULL COMPLETIONS

```
curl -H "Authorization: Bearer $API_KEY" \
  "${BASE}/api/v1/completions"
```

SEND A TEST WEBHOOK TO YOUR ENDPOINT

```
curl -X POST -H "Authorization: Bearer $API_KEY" \
  "{BASE}/api/v1/test-webhook"
```

Uploading your own SCORM lessons for us to host is also available at POST
{BASE}/api/v1/scorm (same Bearer key) — ask Jeremy if you want that turned on.

🔑 Your two credentials (issued by Jeremy, out-of-band)

- **Shared secret** (`INTEGRATION_SECRET`) — signs & verifies **both** launch tokens and webhook signatures. Keep it server-side only.
- **API key** (`INTEGRATION_API_KEY`) — Bearer auth for the pull/reconcile and test endpoints. Rotatable independently of the secret.

These are never printed in this document, the app, or any link. Jeremy provides them directly and can rotate either at any time.

Already built and live. Both directions — the signed launch in and the signed completion webhook out — are running today and were tested end to end. There is no code for GetMatchReady's side to finish; this is the contract for Zite to call.

GetMatchReady × Zite integration reference. Prepared by Jeremy Layton Brown Smith, GetMatchReady. One contract for every organization you connect (OMG/OMS, NCSRA, and beyond) — only the org value and credentials change. Confidential, for the recipient's technical team. The shared secret and API key are issued separately and never appear here.